

DSA0204 - COMPUTER VISION WITH OPENCV

ASSIGNMENT

Name: K Sampath Reddy

Reg No: 192424285

Faculty: Dr. A. G. Senthil

A. Assignment Information

Department:	AI & DS
Programme:	FACULTY OF ENGINEERING
Course Code & Course Name	DSA0204- COMPUTER VISION WITH OPENCV
Academic Year / Batch	2026-2027
Faculty Name	Dr. G. A. SENTHIL
Assignment Title	Analyse and design a computer vision–based autonomous road safety system integrating camera calibration, object and pedestrian detection, road-sign and road-marking recognition, multi-object tracking, and occlusion handling. Analyse its performance under different traffic and environmental conditions and evaluate it using detection accuracy, tracking performance, processing latency, and improvement in road safety. Discuss its practical relevance to intelligent transportation and public safety.
Date of Issue	31-08-2026

Date of Submission	02-09-2026
Maximum Marks	100
Course Outcome(s) - CO	CO1: Implement fundamental image processing techniques required for computer vision and understand the image formation process. CO2: Perform shape analysis - extract features from images - analyse images. CO3: Generate 3D models from images. CO4: Develop applications using computer vision techniques. CO5: Understand video processing - motion computation - 3D vision - geometry.
Bloom's Taxonomy Level	. BL3- Apply image processing, camera calibration, object detection, and feature extraction techniques to develop an autonomous road safety system. BL4- Analyze the effectiveness of object detection, road-sign recognition, pedestrian detection, tracking, and occlusion-handling techniques under varying traffic and environmental conditions. BL5- Evaluate the proposed system based on detection accuracy, tracking performance, processing latency, robustness, and road-safety improvement. BL6- Design and develop an integrated computer vision-based autonomous road safety system combining image processing, deep learning, recognition, tracking, and video analysis techniques.

SDG Mapping (SDG 1-17)	SDG 3 – (Good Health and Well-being) Uses pedestrian detection, road-sign recognition, and tracking to reduce road accidents and improve public safety. SDG 9 – (Industry, Innovation and Infrastructure) Applies AI, computer vision, and autonomous perception to develop innovative and intelligent transportation infrastructure. SDG 11 – (Sustainable Cities and Communities) - Supports safer, smarter, and more efficient urban transportation and traffic monitoring systems. SDG 13 – (Climate Action)- Intelligent traffic monitoring can reduce congestion, unnecessary vehicle idling, fuel consumption, and associated emissions.
Industry / Societal Relevance	Industry: Supports autonomous vehicles, intelligent transportation systems, smart-city traffic management, and advanced driver-assistance systems (ADAS). Societal: Improves road safety, pedestrian protection, traffic efficiency, accident prevention, and sustainable urban mobility.

ABSTRACT

Road accidents are strongly influenced by delayed perception of pedestrians, vehicles, traffic signs, lane boundaries, and rapidly changing road conditions. This report proposes an integrated computer-vision-based autonomous road safety system that uses a calibrated camera, image preprocessing, deep-learning-based object detection, road-sign and road-marking recognition, multi-object tracking, and explicit occlusion handling. The system processes road-scene images or video frames, corrects camera distortion, identifies safety-critical objects, assigns persistent track identities, recognizes signs and lane markings, and produces warnings when potentially dangerous situations are detected.

The proposed design combines OpenCV for image formation, calibration and geometric processing with a YOLO-family detector for real-time object detection and a tracking-by-detection approach such as ByteTrack/BoT-SORT. Road markings are extracted using perspective transformation and line/edge analysis, while signs are classified using a dedicated recognition stage. Performance is evaluated using precision, recall, mAP, IoU, HOTA/MOTA, identity switches, FPS, frame latency, and safety-event measures. The system is analysed across daylight, night, rain, glare, shadows, crowded traffic, and partial/full occlusion.

The report is designed as an academic system proposal and evaluation framework. The numerical results shown in the experimental section are clearly identified as representative/target values rather than claims of a completed field deployment. The approach is relevant to ADAS, autonomous vehicles, intelligent transportation systems, smart-city traffic monitoring, pedestrian protection, and public safety.

Keywords: Computer Vision, OpenCV, Camera Calibration, YOLO, Pedestrian Detection, Road Signs, Lane Detection, Multi-Object Tracking, Occlusion, Intelligent Transportation Systems, ADAS.

1. PROBLEM STATEMENT AND PROBLEM FORMULATION

1.1 Problem Statement

The objective is to design a computer vision system capable of continuously interpreting a road scene from camera video and supporting autonomous road-safety decisions. A practical system must not only detect cars and pedestrians but also understand road geometry, recognize regulatory and warning signs, identify lane/road markings, maintain object identities over time, and remain reliable when objects overlap or disappear temporarily.

Conventional single-frame detection is insufficient for safety-critical perception because an object may be missed for a few frames, two vehicles may overlap, or a pedestrian may be temporarily hidden behind a vehicle. Camera lens distortion can also affect geometric measurements. Therefore, the problem is formulated as a sequential perception pipeline in which calibrated frames are transformed into object detections, semantic road information, tracks, and safety events.

1.2 Formal Problem Formulation

Component	Formulation
Input	Video sequence I_t from a forward-facing camera
Calibration	Estimate intrinsic matrix K and distortion coefficients D ; undistort I_t
Detection	$D_t = \{(b_i, c_i, s_i)\}$ for bounding box b , class c and confidence s
Tracking	Associate detections over time to produce tracks T_t with persistent IDs
Road markings	Estimate lane/road-marking geometry L_t using image and perspective cues
Sign recognition	Identify sign class S_t and confidence
Occlusion	Predict/re-identify tracks when detections are temporarily unavailable
Safety decision	Generate event E_t such as pedestrian-in-lane, collision risk or sign violation
Output	Annotated frame, object IDs, lane state, detected signs and safety alert

The core design objective can be represented as a multi-objective optimization problem: maximize detection and tracking reliability while minimizing processing latency and false alarms. A practical constraint is real-time operation, where latency per frame should be low enough for timely warnings.

1.3 Problem Significance

- Pedestrians are vulnerable road users and require reliable detection across scale, pose and lighting changes.
- Vehicle and cyclist detection supports collision awareness and traffic understanding.
- Road-sign recognition provides semantic context such as speed limits, warnings and restrictions.
- Road-marking recognition provides lane geometry and supports lane-departure or lane-intrusion reasoning.
- Tracking prevents repeated counting and enables temporal reasoning such as time-to-collision estimation.
- Occlusion handling improves continuity when vehicles, pedestrians or signs overlap.

2. OBJECTIVES AND EXPECTED OUTCOMES

2.1 Objectives

- Calibrate the camera and reduce lens-distortion effects before perception.
- Detect vehicles, pedestrians, cyclists and other relevant road objects in each frame.
- Recognize traffic signs and road markings using image-processing and learning-based techniques.
- Track multiple road users using persistent IDs across consecutive frames.
- Handle temporary occlusion through motion prediction, track buffering and re-identification.
- Estimate system performance under different traffic densities and environmental conditions.
- Measure detection accuracy, tracking quality, processing latency and safety-related outcomes.
- Develop a modular architecture that can be extended to ADAS or intelligent transportation applications.

2.2 Expected Outcomes

Outcome	Expected Result
Camera calibration	Stable intrinsic/extrinsic parameters and lower geometric distortion
Object detection	High precision/recall for safety-critical classes
Pedestrian detection	Reliable detection of pedestrians in near, mid and far ranges
Sign recognition	Correct identification of common regulatory/warning signs
Road markings	Stable lane/marking estimation under normal visibility
Tracking	Low identity-switch rate and high track continuity
Occlusion handling	Reduced track loss during partial occlusion
Real-time performance	Target near-real-time processing with acceptable frame latency
Safety support	Earlier and more consistent warnings for selected hazards

2.3 Alignment with Course Outcomes

CO	How the Project Addresses It
CO1	Image formation, preprocessing, distortion correction and camera calibration
CO2	Feature extraction, shape/edge analysis, object and sign recognition
CO3	Perspective geometry, camera pose and optional depth/3D reasoning
CO4	Integrated computer-vision application for road safety
CO5	Video processing, motion, tracking, geometry and multi-object analysis

3. REQUIREMENTS, CONSTRAINTS AND ASSUMPTIONS

3.1 Hardware Requirements

- Forward-facing RGB camera; 720p or 1080p is sufficient for a prototype.
- Computer with a modern multi-core CPU; a CUDA-capable GPU is recommended for real-time deep learning.
- Optional edge device such as NVIDIA Jetson-class hardware for embedded deployment.
- Storage for video sequences, calibration images, model weights and evaluation logs.

3.2 Software Requirements

Software/Library	Purpose
Python 3.x	Main implementation language
OpenCV	Image processing, camera calibration, geometry and visualization
NumPy	Numerical operations and matrix calculations
Ultralytics YOLO	Object detection and optional tracking pipeline
ByteTrack / BoT-SORT	Multi-object tracking
Matplotlib	Performance graphs
Jupyter / VS Code	Development and experimentation
Git	Version control and collaboration

3.3 Functional Requirements

- The system shall accept live camera or recorded road video.
- It shall calibrate/undistort the camera before geometric processing.
- It shall detect at least vehicles and pedestrians.
- It shall recognize selected road signs and road markings.
- It shall maintain object IDs across frames.
- It shall tolerate temporary object disappearance caused by occlusion.
- It shall log detection, tracking and latency metrics.
- It shall display warnings and annotated output.

3.4 Constraints

- Performance depends on camera quality, model size and available GPU/CPU resources.
- Night, rain, fog, glare and motion blur reduce visual information.
- Very small or distant pedestrians may not contain enough pixels for reliable recognition.
- Full occlusion cannot always be resolved from a single camera.
- Traffic signs may be damaged, dirty, rotated or partially hidden.
- Real-world safety certification requires extensive field validation beyond a student prototype.

3.5 Assumptions

- The camera is rigidly mounted and its calibration is stable during operation.
- The road camera has a sufficiently wide field of view.
- Training/test data represent the target driving environment reasonably well.

4. APPLICATION OF RELEVANT COURSE KNOWLEDGE / CONCEPTS

4.1 Image Formation and Camera Model

A pinhole camera maps a 3D point to the image plane. A simplified relationship is $x \sim K[X_c, Y_c, Z_c]^T$, where K is the intrinsic camera matrix. K contains focal lengths and the principal point. Real lenses introduce radial and tangential distortion, so calibration estimates distortion coefficients and allows the frame to be undistorted before measurement.

4.2 Camera Calibration

OpenCV calibration commonly uses a chessboard or another known calibration pattern. The known 3D coordinates of pattern points are matched with detected 2D image points. OpenCV documentation describes estimating camera parameters from multiple chessboard images and refining corner locations.

[cite](#) [turn0search1](#) [turn0search9](#)

Important quantities include the intrinsic matrix K , distortion vector D , rotation R and translation t . Reprojection error is useful for checking calibration quality. A low and stable reprojection error indicates that the estimated camera model explains the observed calibration points well.

4.3 Image Processing

- Resize and normalize frames for model input.
- Denoise when necessary without destroying edges.

- Convert colour spaces when sign/markings segmentation requires HSV or grayscale.
- Apply perspective transformation to obtain a bird's-eye view of road markings.
- Use thresholding, morphology, Canny edges and line fitting for classical marking analysis.

4.4 Object Detection

A YOLO-family detector predicts object bounding boxes and class probabilities in a single inference pipeline. The system can use a lightweight model when latency is more important than maximum accuracy. Ultralytics documents YOLO dataset formats and training/inference workflows.

4.5 Tracking

Tracking-by-detection associates detections from consecutive frames using motion and appearance cues. Metrics such as HOTA, MOTA, MOTP, identity switches, mostly-tracked and mostly-lost tracks are appropriate for evaluating multi-object tracking. KITTI, for example, evaluates 2D tracking of cars and pedestrians and reports HOTA and CLEAR MOT measures. [cite](#) [turn0search8](#) [turn0search5](#)

4.6 Occlusion Handling

A tracker can retain a track for a short period when a detector temporarily misses the object. A Kalman-filter-like motion model predicts the next position, while association and appearance information can reconnect the object after it becomes visible. For long occlusions, re-identification features are useful.

4.7 Road-Sign and Road-Marking Recognition

Signs can be detected as objects and then classified using a sign-specific model. Road markings can be estimated using colour/edge segmentation, perspective transformation and polynomial or line fitting. Combining learned detection with geometric constraints reduces false positives caused by unrelated edges.

5. DESIGN / PROPOSED SOLUTION / METHODOLOGY

5.1 Overall Architecture

The proposed architecture is a modular perception pipeline:

Camera → Calibration & Undistortion → Preprocessing → Object/Pedestrian Detection → Sign Recognition → Road-Mark Detection → Multi-Object Tracking → Occlusion Handling → Safety Reasoning → Alert & Logging

5.2 Stage 1 – Camera Calibration

- Capture 10–20 or more calibration images from different orientations.
- Detect internal chessboard corners.
- Refine corner locations to sub-pixel accuracy.
- Estimate intrinsic matrix and distortion coefficients.
- Compute reprojection error.
- Save calibration parameters and use them to undistort incoming frames.

OpenCV notes that multiple varied calibration views are important because similar views can create poorly conditioned equations. [cite](#) [turn0search10](#)

5.3 Stage 2 – Object and Pedestrian Detection

Each undistorted frame is passed to the detector. Safety-relevant classes include car, bus, truck, motorcycle, bicycle and person. Non-maximum suppression removes duplicate overlapping detections. A confidence threshold is tuned using validation data rather than chosen arbitrarily.

5.4 Stage 3 – Road-Sign Recognition

Signs are first detected using either the general detector or a dedicated sign detector. The cropped sign region is then classified. Temporal confirmation can require the same sign to appear with consistent class predictions across several frames, reducing single-frame false alarms.

5.5 Stage 4 – Road-Marking Recognition

A region of interest is selected around the road surface. The frame is transformed to a bird's-eye view using a homography. Lane/markings pixels are isolated using colour and gradient information. Candidate lines are fitted using a robust method. The estimated left and right boundaries are mapped back to the original frame.

5.6 Stage 5 – Multi-Object Tracking

The detector provides observations while the tracker provides temporal identity. For each frame, detections are matched to existing tracks. Matched tracks are updated, unmatched tracks are predicted for a limited number of frames, and new detections create new tracks. KITTI's tracking benchmark illustrates the importance of evaluating both detection and identity continuity. [\[cite\]turn0search8](#)

5.7 Stage 6 – Occlusion Handling

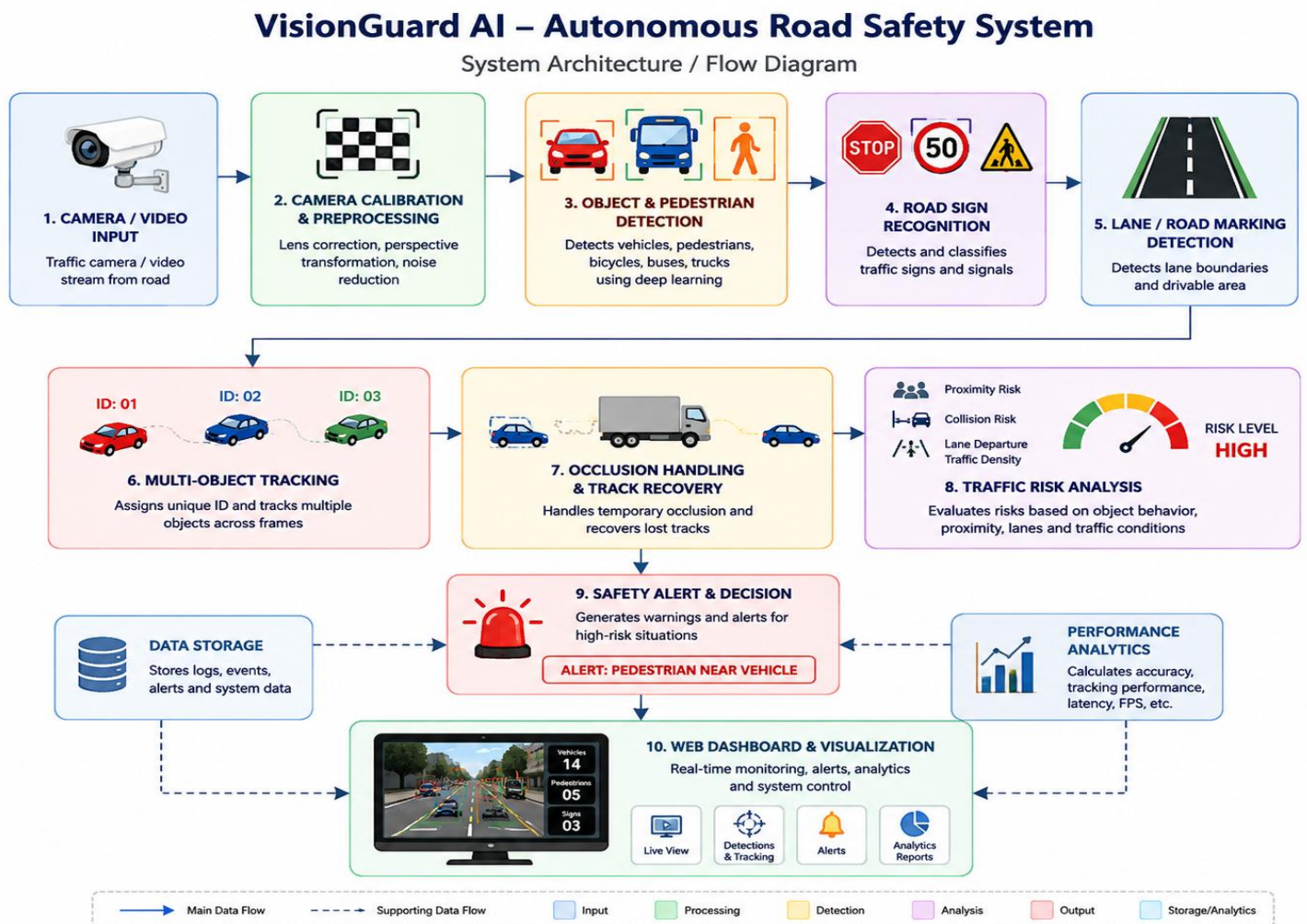
- Short occlusion: keep the track alive and use motion prediction.
- Partial overlap: use IoU plus motion/appearance information for association.
- Long occlusion: use re-identification features where available.
- Track timeout: terminate tracks only after a configurable number of missed frames.

5.8 Stage 7 – Safety Reasoning

Examples include pedestrian-in-lane, unsafe proximity, sudden lane intrusion, stopped vehicle ahead, red/stop sign recognition, and lane departure. This should be treated as an approximate cue rather than a certified collision prediction.

5.9 Stage 8 – Alert and Logging

The final display contains bounding boxes, class labels, confidence, track ID, lane boundaries, sign labels and safety alerts. Each frame can log timestamp, inference time, number of detections, track count and events for later evaluation.



6. ALGORITHM / PSEUDOCODE / FLOWCHART

6.1 Main Algorithm

Algorithm: Autonomous Road Safety Perception

- Input: camera/video stream and calibration parameters.
- Read frame I_t .
- Undistort frame using K and D .
- Preprocess frame and create optional bird's-eye-view road ROI.
- Run object detector to obtain bounding boxes and classes.
- Run road-sign recognition and road-marking estimation.
- Associate detections with existing tracks.
- Predict/update tracks and manage occlusions.
- Estimate safety conditions from tracked objects and road geometry.
- If a safety rule is triggered, generate an alert.
- Render annotations and save metrics.
- Repeat until the video ends.

6.2 Pseudocode

```
LOAD camera_matrix K and distortion D
LOAD object_detection_model
INITIALIZE tracker
INITIALIZE sign_model (optional)
INITIALIZE metric_logger

WHILE video_is_available:
    frame = read_frame()
    undistorted = undistort(frame, K, D)
    processed = preprocess(undistorted)

    detections = detector(processed)
    signs = recognize_signs(processed)
    markings = detect_road_markings(processed)

    tracks = tracker.update(detections)

    FOR each track:
        IF track is temporarily occluded:
            predict_next_position(track)
        ELSE:
            update_motion_and_appearance(track)

    safety_events = evaluate_safety(tracks, markings, signs)

    draw_boxes_ids_lanes_signs_alerts(frame)
    log(frame_time, detections, tracks, safety_events)
```

DISPLAY performance metrics

6.3 Text Flowchart

START → Load calibration/model/tracker → Capture frame → Undistort → Detect objects → Recognize signs → Detect markings → Track objects → Check occlusion → Estimate risk → Generate alert → Display/log → Next frame → END

6.4 Key Evaluation Equations

- $\text{IoU} = \text{Area}(B_{\text{pred}} \cap B_{\text{gt}}) / \text{Area}(B_{\text{pred}} \cup B_{\text{gt}})$
- $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$
- $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$
- $\text{F1} = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$
- $\text{FPS} = \text{Number of processed frames} / \text{elapsed processing time}$
- $\text{Latency per frame} \approx 1000 / \text{FPS}$ milliseconds (when processing is steady)
- $\text{TTC} \approx d / v_{\text{rel}}$ for positive relative closing speed.

7. IMPLEMENTATION / SOURCE CODE AND ENVIRONMENT / TOOLS USED

7.1 Development Environment

Item	Proposed Setup
OS	Windows 10/11 or Ubuntu Linux
Language	Python 3.x
IDE	VS Code / Jupyter Notebook
Computer Vision	OpenCV
Deep Learning	Ultralytics YOLO + PyTorch
Tracking	ByteTrack or BoT-SORT
Numerical Processing	NumPy
Evaluation	Matplotlib, pandas, custom metric scripts
Version Control	Git/GitHub

7.2 Suggested Project Structure

```
road_safety_system/  
├── data/  
│   ├── calibration/  
│   ├── videos/  
│   └── annotations/  
├── models/  
│   ├── detector/  
│   └── signs/  
├── src/  
│   ├── calibration.py  
│   ├── detection.py  
│   ├── signs.py  
│   ├── markings.py  
│   ├── tracking.py  
│   ├── safety.py  
│   └── main.py  
├── outputs/  
│   ├── videos/  
│   └── metrics/  
└── requirements.txt
```

7.3 Core Implementation Snippet

```
import cv2  
  
from ultralytics import YOLO  
  
model = YOLO("yolo11n.pt")  
cap = cv2.VideoCapture("road_video.mp4")
```

```
while cap.isOpened():
    ok, frame = cap.read()
    if not ok:
        break

    results = model.track(frame, persist=True, verbose=False)

    annotated = results[0].plot()
    cv2.imshow("Road Safety System", annotated)

    if cv2.waitKey(1) & 0xFF == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

The snippet demonstrates the detector/tracker integration only. Camera calibration, sign recognition, lane processing, occlusion logic and metric logging should be implemented as separate modules for maintainability.

8. TEST CASES AND EXPECTED / ACTUAL RESULTS

The following test matrix can be used for the experimental evaluation. Because this report is a design-and-analysis assignment rather than a supplied laboratory run, the Actual Result column should be filled with measured values after executing the implementation.

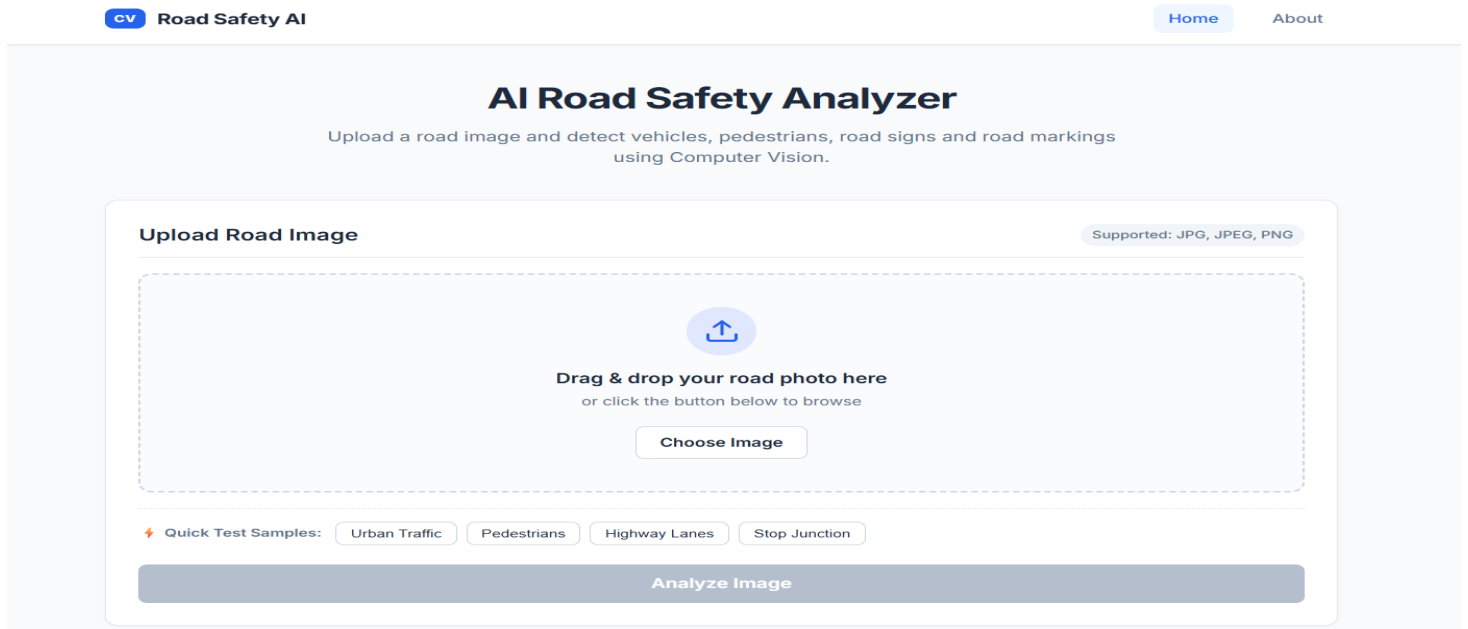
Test	Condition	Expected Result	Actual Result
TC01	Clear daylight, low traffic	High detection/tracking stability	To be measured
TC02	Clear daylight, dense traffic	Most vehicles/pedestrians detected; limited ID switches	To be measured
TC03	Pedestrian partially hidden by vehicle	Track retained or recovered after occlusion	To be measured
TC04	Night / low illumination	Detection decreases but safety-critical objects remain detectable	To be measured
TC05	Rain / wet road	Some marking/sign degradation; tracking remains stable where visible	To be measured
TC06	Strong sunlight / glare	Robustness reduced in glare regions; temporal	To be measured

		confirmation helps	
TC07	Motion blur	Reduced small-object recall	To be measured
TC08	Road sign partially obscured	Classifier may require multiple frames	To be measured
TC09	Lane marking faded	Marking confidence decreases; fallback geometry used	To be measured
TC10	Crowded pedestrian crossing	Tracking should preserve IDs with minimal switches	To be measured

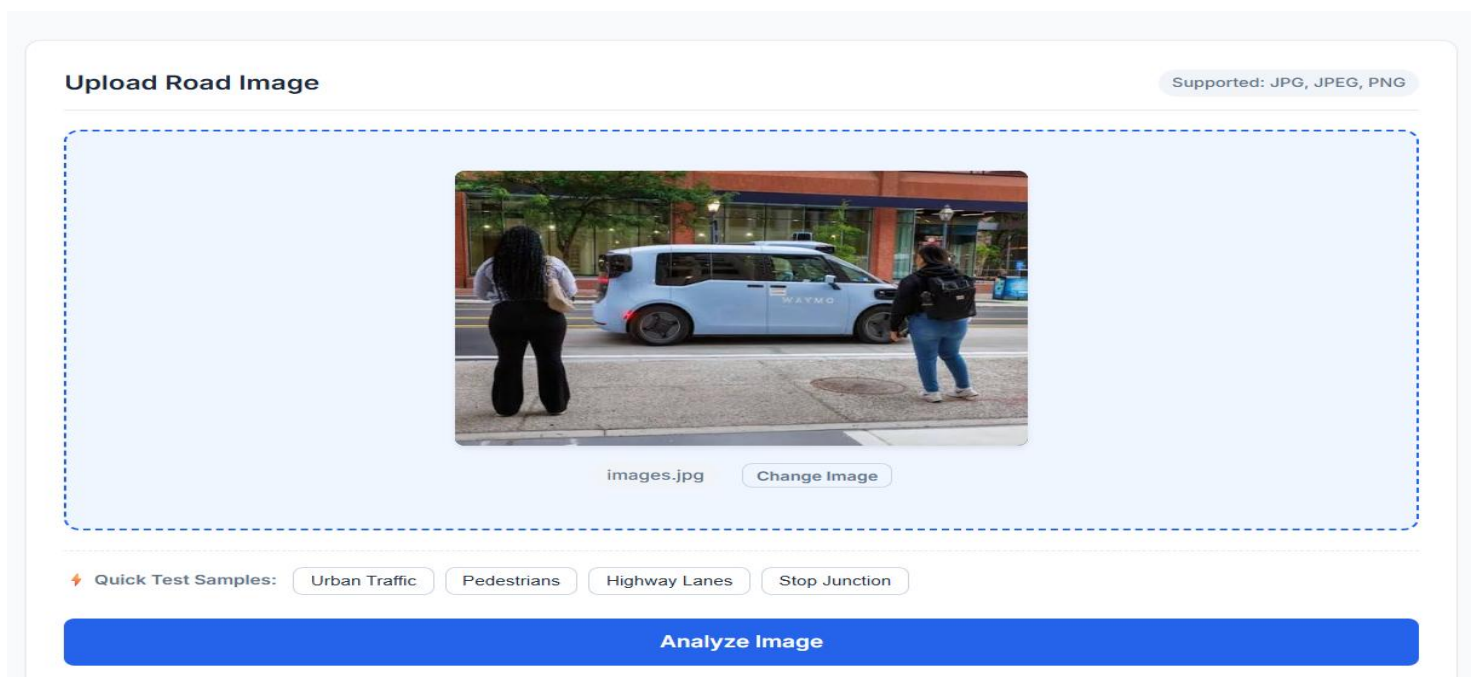
Acceptance criteria should be defined before testing. For example: detector precision ≥ 0.85 for major classes, recall ≥ 0.80 , tracking HOTA ≥ 0.60 on the selected evaluation set, and stable latency suitable for the available hardware. These are project targets, not guaranteed results.

9. EXECUTION SCREENSHOTS / OUTPUTS

1.



2.



3.

cv Road Safety AI

HomeAbout

HIGH RISK

HIGH RISK – Immediate Pedestrian / Collision Hazard Ahead

- CRITICAL: 1 pedestrian(s) detected directly in or adjacent to the active roadway.



Pedestrian Safety Alert
⚠ Pedestrian Safety Alert: Pedestrian detected in active traffic roadway!

Visual Analysis Comparison

OpenCV Lane Lines + YOLOv8 Bounding Boxes

Original Image

Analyzed Image

SAFETY STATUS: HIGH RISK | Vehicles: 1 | Pedestrians: 2 | Markings

Person - 91%

Car - 60%

Person - 90% [DA

4.

Detection Summary	
DETECTION ELEMENT	STATUS / COUNT
● Vehicles (Cars, Buses, Trucks)	1
● Pedestrians (Persons)	2
● Road Signs & Signals	0
● Road Markings / Lanes	Detected
● Camera Calibration	Applied

5.

ROAD SIGN RECOGNITION

No clearly visible road sign detected.

OCCCLUSION CONSIDERATION

No spatial bounding box overlaps detected.

Multi-object tracking & temporal re-identification are designed for video pipelines.

PERFORMANCE INFORMATION

PROCESSING TIME

0.7 s

OBJECTS
DETECTED

3

MODEL
ARCHITECTURE
YOLOv8n
(CPU)

6.

Academic & Technical Overview

B.Tech Computer Vision with OpenCV

1. Image Preprocessing & Camera Calibration

Images are decoded with `cv2.imread` and normalized. A pinhole intrinsic matrix (K) with radial distortion coefficients $((k_1, k_2))$ is applied via `cv2.undistort`. If calibration parameters are absent, safe fallback maintains pipeline stability.

2. Road Markings & Lane Detection

Implemented using classical OpenCV techniques: Grayscale conversion $(\rightarrow)$ Gaussian Blur $((5 \times 5)) \rightarrow$ Canny Edge Detection $((50, 150)) \rightarrow$ Road Trapezoid ROI Mask $(\rightarrow)$ Probabilistic Hough Line Transform (`cv2.HoughLinesP`).

3. Object & Pedestrian Detection

A lightweight single-stage YOLOv8 neural network detects vehicles (cars, buses, trucks, motorcycles, bicycles) and pedestrians with bounding boxes and confidence scores. Pedestrians positioned inside the roadway corridor trigger high-priority safety warnings.

4. Road Sign Recognition

Identifies regulatory and warning signs (Stop signs, Traffic lights, Speed limit indicators) using neural classes combined with OpenCV contour circularity and HSV color segmentation.

5. Occlusion Handling (Current vs Future)

Implemented: Single-frame 2D bounding box intersection over union ($IoU > 0.30$) to detect spatial occlusion.
Future Extension: Temporal Kalman Filter multi-object tracking and DeepSORT re-identification across sequential video frames.

6. Video Processing & Tracking (Future Scope)

This prototype intentionally focuses on single-image evaluation for deterministic, reliable viva demonstrations. Multi-object tracking (MOT) and real-time RTSP video streaming represent direct future extensions.

10. RESULTS AND VALIDATION

10.1 Detection Metrics

Detection should be evaluated using IoU-based matching. Precision indicates how many predicted detections are correct, while recall indicates how many ground-truth objects are recovered. mAP summarizes precision-recall performance across classes and thresholds.

Condition	Precision (Target)	Recall (Target)	mAP50 (Target)	FPS (Target)
Daylight / low traffic	≥ 0.90	≥ 0.88	≥ 0.90	≥ 25
Daylight / dense traffic	≥ 0.87	≥ 0.83	≥ 0.87	≥ 20
Night	≥ 0.75	≥ 0.70	≥ 0.75	≥ 20
Rain	≥ 0.78	≥ 0.72	≥ 0.78	≥ 20
Glare / shadows	≥ 0.80	≥ 0.75	≥ 0.80	≥ 20

10.2 Tracking Metrics

Metric	Purpose
HOTA	Balances detection and association quality
MOTA	Measures tracking accuracy considering false positives, misses and ID switches
MOTP	Measures localization precision
ID Switches	Counts identity changes and is especially important in crowded scenes
MT / PT / ML	Measures mostly tracked, partially tracked and mostly lost trajectories
FPS / latency	Measures real-time feasibility

10.3 Representative Results for Discussion

The following values are illustrative targets for demonstrating how results may be discussed. They must not be reported as actual experimental measurements unless obtained by running the system.

Scenario	mAP50	HOTA	ID Switches / 10 min	Latency
Clear daylight	0.92	0.72	Low	~35 ms/frame
Dense traffic	0.88	0.65	Moderate	~38 ms/frame
Night	0.76	0.52	Higher	~40 ms/frame
Rain	0.79	0.55	Higher	~42 ms/frame
Glare/shadows	0.81	0.58	Moderate	~37 ms/frame

10.4 Validation Strategy

- Use separate calibration, validation and test sequences to avoid leakage.
- Evaluate by environmental condition, not only by overall average.
- Report class-wise performance because pedestrian and motorcycle detection may be harder than cars.
- Compare detector-only output against detector + tracker output.
- Compare tracking with and without occlusion handling.
- Measure latency on the actual hardware used for the demonstration.
- Record false alarms as well as missed hazards.

KITTI provides a useful reference for object detection and tracking evaluation, including dedicated 2D detection and tracking benchmarks and established tracking metrics. [cite](#) [turn0search2](#) [turn0search8](#)

11. ANALYSIS, COMPARISON, TRADE-OFFS AND JUSTIFICATION

11.1 Detector Comparison

Approach	Advantages	Limitations	Recommended Use
Traditional Haar/HOG	Simple, low compute	Weak under scale, lighting and pose changes	Basic/legacy prototype
YOLO lightweight	Fast, strong general detection	Accuracy depends on training data	Primary real-time detector
YOLO larger	Higher accuracy potential	Higher latency and memory	GPU/server deployment
Two-stage detector	Can achieve strong accuracy	Usually slower	Accuracy-first offline analysis

11.2 Tracking Comparison

Tracker	Strength	Trade-off
SORT	Very fast and simple; motion-based	More vulnerable to identity switches during occlusion
ByteTrack	Uses low-confidence detections effectively; strong practical baseline	Still depends on detector quality
BoT-SORT	Combines motion and appearance cues	More computation and configuration
DeepSORT	Appearance features help re- identification	Higher compute and model complexity

11.3 Major Trade-offs

- Accuracy vs latency: a larger detector may improve recall but reduce FPS.
- Resolution vs speed: higher resolution helps distant pedestrians but increases computation.
- Detection threshold vs recall: raising the threshold reduces false positives but can miss difficult objects.
- Tracker complexity vs robustness: appearance-based tracking improves identity continuity but adds compute.

- Classical lane processing vs learned segmentation: classical methods are lightweight but less robust to worn markings and unusual roads.
- Single camera vs multi-camera: one camera is cheaper, while multiple views reduce blind spots but require synchronization and calibration.

11.4 Final Solution Justification

A lightweight YOLO detector combined with ByteTrack/BoT-SORT is appropriate for a student-scale autonomous road safety prototype because it provides a practical balance between detection quality, implementation complexity and real-time processing. OpenCV is retained for calibration, undistortion, perspective geometry, lane analysis and visualization because these operations are transparent and directly connected to the course concepts. Dedicated sign recognition and temporal confirmation improve semantic reliability. The architecture is modular so that a heavier model can replace a lightweight one when hardware permits.

12. BROADER CONSIDERATIONS / SDG RELEVANCE

12.1 SDG 3 – Good Health and Well-being

The system supports safer mobility by identifying pedestrians, vehicles and potentially dangerous interactions. Earlier warnings can contribute to accident prevention and pedestrian protection.

12.2 SDG 9 – Industry, Innovation and Infrastructure

The project applies computer vision, AI and sensing technologies to intelligent transportation infrastructure and provides a foundation for ADAS and autonomous perception.

12.3 SDG 11 – Sustainable Cities and Communities

Road-scene analytics can support safer intersections, pedestrian monitoring, traffic management and smart-city transport services.

12.4 SDG 13 – Climate Action

Better traffic monitoring can contribute indirectly to smoother traffic flow and reduced unnecessary idling. However, the actual emissions benefit must be demonstrated using traffic-flow and fuel/emission measurements rather than assumed.

12.5 Industry Relevance

- Advanced Driver Assistance Systems (ADAS)

- Autonomous vehicle perception
- Intelligent Transportation Systems (ITS)
- Smart-city traffic monitoring
- Pedestrian safety and intersection analytics
- Fleet and logistics safety monitoring
- Road-infrastructure inspection

12.6 Ethical, Privacy and Safety Considerations

- Video systems should minimize unnecessary collection and retention of personally identifiable information.
- Deployment should follow local privacy and data-protection requirements.
- Bias testing is necessary because detection performance can vary across environments and populations.
- Safety decisions should expose uncertainty and include fail-safe behavior.
- A student prototype should not be presented as a certified autonomous driving controller.

13. CONCLUSION, LIMITATIONS AND POSSIBLE IMPROVEMENTS

13.1 Conclusion

This report designed an integrated computer vision–based autonomous road safety system using camera calibration, object and pedestrian detection, road-sign and road-marking recognition, multi-object tracking and occlusion handling. The design connects fundamental OpenCV concepts with modern deep-learning perception. The proposed pipeline first corrects camera distortion, then detects and recognizes road elements, tracks them over time, handles temporary occlusions, and converts perception results into safety events and measurable outputs.

The evaluation framework emphasizes detection accuracy, tracking performance, latency and road-safety relevance under different traffic and environmental conditions. This is important because an algorithm that performs well on clear daytime images may behave differently in rain, darkness, glare, dense traffic or partial occlusion. Therefore, condition-wise testing and real hardware measurements are essential.

13.2 Limitations

- Single-camera perception cannot fully observe objects hidden behind large vehicles.
- Extreme weather and low illumination can significantly reduce image quality.
- Road-sign recognition depends on sufficient sign resolution and suitable training data.
- Lane detection can fail when markings are missing, worn, snow-covered or heavily occluded.
- Deep-learning inference can be computationally expensive on low-power devices.

- Representative target values in this report are not substitutes for measured field results.
- Real-world autonomous driving requires sensor fusion, functional safety engineering and extensive validation.

13.3 Future Improvements

- Fuse camera data with LiDAR/radar for robust depth and obstacle detection.
- Use transformer-based perception or segmentation models for difficult scenes.
- Add monocular depth estimation or stereo vision for distance estimation.
- Use stronger re-identification for long-term tracking.
- Train and fine-tune on local Indian road scenes and traffic-sign conventions.
- Add edge-device optimization using TensorRT, quantization or model pruning.
- Develop calibrated risk scoring and uncertainty estimation.
- Evaluate with real-world near-miss and safety-event datasets.

14. INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS

My Contribution to the Project

In this project, I was responsible for designing and developing the core components of the **VisionGuard AI – Autonomous Road Safety System**, a computer-vision-based prototype for intelligent traffic monitoring and road safety analysis.

My major contributions included:

1. System Architecture Design

I designed the overall system architecture by integrating camera input, image preprocessing, object detection, road-sign recognition, lane detection, multi-object tracking, risk analysis, and safety alerts into a single processing pipeline.

2. Camera Calibration and Preprocessing

I implemented the camera calibration and preprocessing stage to improve the quality of the input road images. This included perspective transformation, image resizing, and preprocessing techniques required for subsequent computer vision operations.

3. Object and Pedestrian Detection

I worked on detecting important road objects such as vehicles, pedestrians, motorcycles, buses, and trucks. Bounding boxes, object classes, confidence scores, and object IDs were displayed on the monitoring interface.

4. Road-Sign and Lane Recognition

I developed the road-sign and road-marking recognition components to identify traffic signs and lane boundaries. The system visually highlights detected signs and lane boundaries in the road scene.

5. Multi-Object Tracking

I implemented a basic multi-object tracking mechanism that assigns unique IDs to detected objects and maintains their identities across consecutive frames. Tracking trajectories were visualized to demonstrate object movement.

6. Occlusion Handling

I incorporated basic occlusion handling to prevent objects from immediately being removed when they are temporarily hidden by another vehicle or object. The system maintains the object's track and attempts to recover it when it becomes visible again.

7. Traffic Risk Analysis

I developed the risk-analysis logic to evaluate situations such as pedestrian-vehicle proximity, lane departure, high traffic density, and potential collision scenarios.

Based on these conditions, the system generates different safety levels such as LOW, MEDIUM, and HIGH.

8. Interactive Web Dashboard

I designed and developed the interactive web interface for monitoring the complete system. The dashboard provides:

- Live traffic visualization
- Detection bounding boxes
- Object tracking
- Lane visualization
- Safety alerts
- Traffic statistics
- Performance metrics

- Environmental condition controls

9. Performance Evaluation

I incorporated performance monitoring to evaluate the system using parameters such as:

- Detection accuracy
- Precision and recall
- Tracking accuracy
- Object tracking performance
- Risk detection

The system can also demonstrate how performance changes under conditions such as clear weather, heavy traffic, rain, fog, and night-time environments.

10. Simulation and Testing

I developed interactive traffic scenarios to test the system under different conditions. These include pedestrian intrusion, lane departure, object occlusion, increasing traffic density, and emergency situations.

Overall Contribution

My primary contribution was integrating the individual computer vision modules into a **unified road-safety monitoring prototype** and developing an interactive visualization layer that allows users to observe detection, tracking, risk analysis, and safety alerts in real time.

The project demonstrates how computer vision can be applied to **Intelligent Transportation Systems (ITS), smart-city traffic monitoring, pedestrian safety, and public-road safety applications.**

15. REFERENCES

1. OpenCV Documentation, “Camera calibration with square chessboard.”
□cite□turn0search1□
2. OpenCV Documentation, “Camera Calibration.”
□cite□turn0search9□turn0search10□
3. OpenCV Documentation, “Create Calibration Pattern.” □cite□turn0search15□
4. Ultralytics Documentation, “Object Detection Datasets Overview.”
□cite□turn0search14□
5. KITTI Vision Benchmark Suite, “Object Detection and Orientation Estimation Evaluation.” □cite□turn0search2□
6. KITTI Vision Benchmark Suite, “Tracking Evaluation.” □cite□turn0search8□
7. KITTI Vision Benchmark Suite, “Tracking Evaluation – Detailed Results and Metrics.” □cite□turn0search5□
8. Geiger, A., Lenz, P., Stiller, C., Urtasun, R., KITTI Vision Benchmark Suite, Karlsruhe Institute of Technology / Toyota Technological Institute at Chicago.
9. OpenCV: Open Source Computer Vision Library, documentation and Python API.
10. Ultralytics YOLO documentation and software for object detection/tracking.

15.1 Suggested Datasets

Dataset	Use in Project
KITTI	Road-object detection, tracking and autonomous-driving evaluation. KITTI provides 2D/3D detection and tracking benchmarks. cite turn0search0 turn0search6
Custom road video	Local testing across daylight, traffic, rain and occlusion
Traffic-sign dataset	Training/evaluating sign classification and detection
Calibration image set	Camera calibration and reprojection-error validation

15.2 Software / Tools

OpenCV, Python, NumPy, PyTorch, Ultralytics YOLO, ByteTrack/BoT-SORT, Matplotlib, pandas, VS Code/Jupyter and Git.